■ Cherry Rewards

Deployment & Setup Guide

This guide walks you through publishing Cherry Rewards to GitHub and deploying the backend to Render.com with a free managed Postgres database.

---

1. Repository Files

Your zip contains the following files — push them all to GitHub as-is:

File	Purpose
`cherry_backend.py`	Flask backend — REST API + MQTT client
`cherry-rewards.html`	Frontend — single HTML file, no build step needed
`geotags.txt`	Store locations, auto-seeded into Postgres on first boot
`requirements.txt`	Python dependencies for pip
`render.yaml`	Render Blueprint — provisions web service + Postgres together
`.env.example`	Template for local environment variables
`.gitignore`	Excludes .env, __pycache__, venv, etc.
`README.md`	Full instructions (mirrors this guide)

---

2. Push to GitHub

If you don't already have a GitHub repo, create one at github.com/new (keep it public or private — Render supports both). Then run:

```
unzip cherry-rewards.zip
cd cherry-rewards
git init && git add .
git commit -m "Initial Cherry Rewards commit"
git remote add origin https://github.com/YOUR_USERNAME/cherry-rewards.git
git push -u origin main
```

---

3. Deploy on Render.com

Option A — Blueprint (recommended, one-click)

The included **render.yaml** is a Render Blueprint that creates both the web service and a free Postgres database in a single step, with DATABASE_URL wired automatically.

- Go to **dashboard.render.com** → click **New** → **Blueprint**.
- Connect your GitHub account and select the **cherry-rewards** repo.
- Render detects *render.yaml* — review the plan and click **Apply**.
- Wait ~2 minutes for the build to finish. Copy your live URL (e.g. *https://cherry-rewards.onrender.com*).

Option B — Manual setup

- **Web Service:** New Web Service → connect repo → Build command: `pip install -r requirements.txt` · Start command: `gunicorn cherry_backend:app`
- **Database:** New PostgreSQL → free plan → copy the *Internal Database URL*.
- Paste the URL as the `DATABASE_URL` environment variable on your web service (Environment tab).
- Add the remaining environment variables from the table below.

Variable	Value
`DATABASE_URL`	`Render internal Postgres URL (auto-set by Blueprint)`
`MQTT_BROKER`	`broker.hivemq.com`
`MQTT_PORT`	`1883`
`MQTT_USER`	`(leave blank for public HiveMQ)`
`MQTT_PASS`	`(leave blank for public HiveMQ)`
`MQTT_CLIENT_ID`	`cherry_backend_prod`

---

4. Connect the Frontend

After your Render service is live, update the geotag API URL in **cherry-rewards.html**. Find this line near the top of the <script> block:

```
const GEOTAG_API = 'https://your-backend.onrender.com/geotags';
```

Replace the placeholder with your actual Render URL, then commit and push. Render will auto-redeploy within ~60 seconds.

```
git add cherry-rewards.html
git commit -m "Set production GEOTAG_API URL"
git push
```

---

5. What Happens on First Boot

The backend automatically performs these tasks on startup — no manual SQL required:

- Creates `geotags`, `users`, and `rewards` tables (if they don't exist).
- Seeds all stores from **geotags.txt** into the database using `ON CONFLICT DO NOTHING` — safe to redeploy without duplicating.
- Connects to the MQTT broker and subscribes to `cherry/qr_scan` and `cherry/login`.

---

6. Add or Edit Store Locations

Edit **geotags.txt** — one store per line in the format `lat, lng, Store Name`. Lines starting with # are ignored.

```
# Auckland, NZ
-36.8485, 174.7633, Cherry – Auckland City Centre
-36.8600, 174.7500, Cherry – Westfield Newmarket
# Sydney, AU
-33.8688, 151.2093, Cherry – Sydney CBD
```

Commit and push — Render redeploys and seeds the new stores on next boot.

---

7. MQTT Topics

Topic	Direction	Payload
`cherry/qr_scan`	Frontend → Backend	{ data, user, timestamp }
`cherry/login`	Frontend → Backend	{ email, mode, timestamp }
`user_id/rewards`	Backend → Frontend	{ title, desc, exp }
`111aabc/email`	Backend → Frontend	{ user_id, email }

---

8. Verify the Deploy

Once live, test these two endpoints in your browser or with curl:

```
GET https://cherry-rewards.onrender.com/
→ { "status": "ok", "service": "Cherry Rewards Backend" }
```

```
GET https://cherry-rewards.onrender.com/geotags?lat=-36.8485&lng=174.7633&radius=8000
→ [ { "name": "Cherry – Auckland City Centre", "lat": ..., "distance_m": ... }, ... ]
```

---

9. Run Locally

```
python -m venv venv && source venv/bin/activate
pip install -r requirements.txt
cp .env.example .env # then edit DATABASE_URL
python cherry_backend.py
```

Open **cherry-rewards.html** directly in your browser, or serve it with:

```
python -m http.server 8080
```

Then visit **http://localhost:8080/cherry-rewards.html**.
